

Домашняя работа № 1

Методы кодирования и модуляция сигналов

Павличенко Родион Андреевич

2026-09-15

Докладчик

Павличенко Родион Андреевич

- ▶ студент Российского университета дружбы!- народов;
- ▶ группа НПИбд-02-24;
- ▶ электронная почта: 1132246838@rudn.ru;
- ▶ дисциплина: «Сетевые технологии».

Цель работы

Изучить методы построения, кодирования и модуляции сигналов в GNU Octave.

Определить спектр и параметры сигналов, рассмотреть амплитудную модуляцию и исследовать свойства самосинхронизации различных способов кодирования.

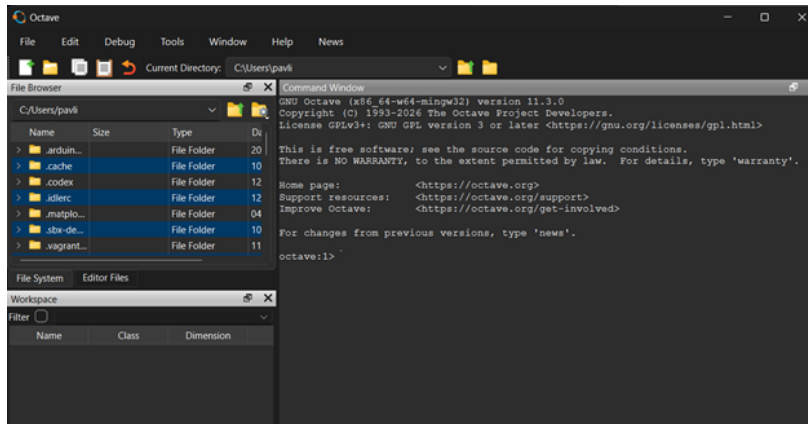
Основные задачи

- ▶ построить графики тригонометрических функций;
- ▶ выполнить разложение меандра в ряд Фурье;
- ▶ определить спектры сигналов;
- ▶ проверить влияние частоты дискретизации;
- ▶ смоделировать амплитудную модуляцию;
- ▶ реализовать несколько способов линейного кодирования;
- ▶ исследовать самосинхронизацию кодов.

Запуск GNU Octave

Для выполнения работы была установлена и запущена среда GNU Octave.

В программе был выбран рабочий каталог для хранения сценариев и полученных изображений.



Создание первого сценария

В редакторе был создан сценарий для построения функции:

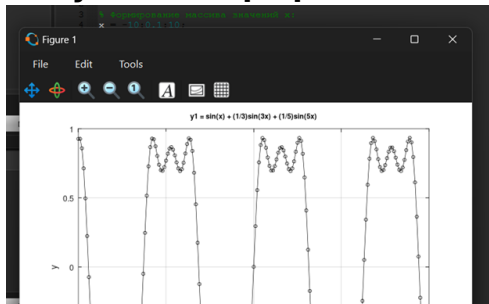
$$y_1 = \sin x + \frac{1}{3} \sin 3x + \frac{1}{5} \sin 5x.$$

После запуска сценария был получен график суммы трёх синусоидальных гармоник.

Программный код

```
1 # Создадим массив данных x
2 x = 10 * 0.1:10;
3 # Создадим массив y
4 y1 = sin(x) + 1/3 * sin(3*x) + 1/5 * sin(5*x);
5 # Создадим график
6 plot(x, y1, 'b-o', 'linewidth',
7      '(1/3)*sin(3*x) + (1/5)*sin(5*x)', 'marker', 'o', 'markerfacecolor', 'b',
8      'linestyle', 'solid', 'color', 'b')
9 grid on
10 # Добавим метки
11 xlabel('x');
12 # Добавим метку y
13 ylabel('y');
14 # Добавим заголовок
15 title('y1 = sin(x) + (1/3)*sin(3*x) + (1/5)*sin(5*x)');
16 # Добавим параметры к графику
17 print('plot-win.png', 'r-mono', '-r-ascii:16', '-dps')
18 # Сохраним график в файл
19 print('plot-win.png');
```

Полученный график



Добавление второй функции

В сценарий была добавлена функция:

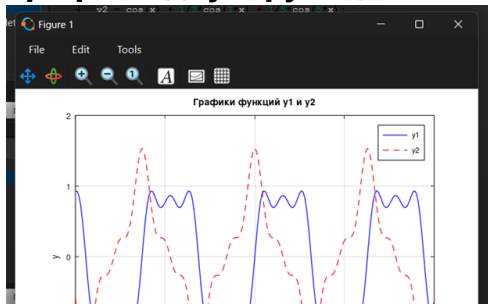
$$y_2 = \cos x + \frac{1}{3} \cos 3x + \frac{1}{5} \cos 5x.$$

Обе функции были построены в одной системе координат линиями разного типа.

Изменённый сценарий

```
Editor
File Edit View Debug Run Help
C:/Octave/plot_sin_cos.m
1
2
3 y1 = sin x + 1/3 sin 3x + 1/5 sin 5x;
4 y2 = cos x + 1/3 cos 3x + 1/5 cos 5x;
5
6 plot x, y1, "b-";
7 hold on;
8 plot x, y2, "r--";
9 hold off;
10
11 grid on;
12 xlabel("x");
13 ylabel("y");
14 title("Графики функций y1 и y2");
15 legend("y1", "y2");
16
17 drawnow;
18 print("C:/Octave/plot_sin_cos.png", "-dpng");
```

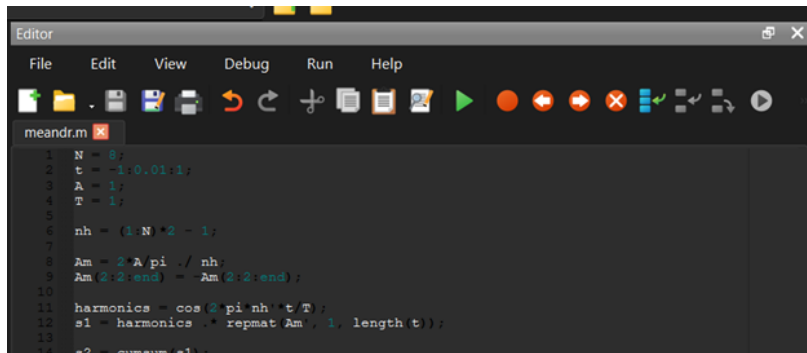
Графики двух функций



Создание сценария меандра

В сценарии `meandr.m` были заданы:

- ▶ количество гармоник;
- ▶ диапазон времени;
- ▶ амплитуда сигнала;
- ▶ период сигнала;
- ▶ коэффициенты ряда Фурье.



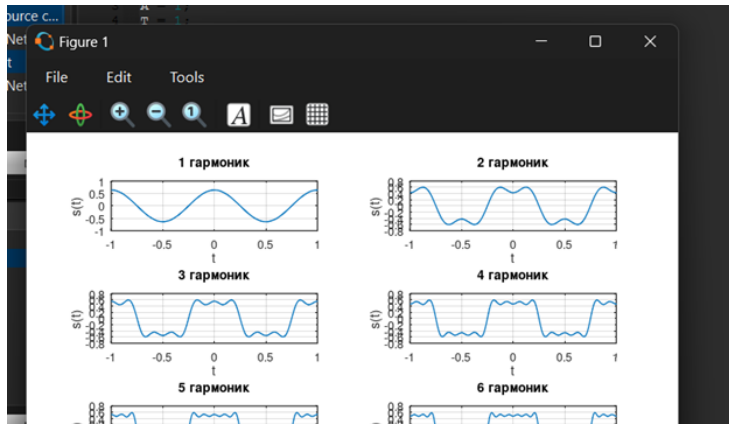
The screenshot shows a MATLAB Editor window with the file `meandr.m` open. The code defines parameters for a meander signal and calculates its harmonics and cumulative sum.

```
1 N = 8;  
2 t = -1:0.01:1;  
3 A = 1;  
4 T = 1;  
5  
6 nh = (1:N)*2 - 1;  
7  
8 Am = 2*A/pi ./ nh;  
9 Am(2:2:end) = -Am(2:2:end);  
10  
11 harmonics = cos(2*pi*nh'*t/T);  
12 s1 = harmonics .* repmat(Am', 1, length(t));  
13  
14 s2 = cumsum(s1);
```

Построение меандра

В результате были получены графики меандра с различным количеством гармоник.

При увеличении количества гармоник форма сигнала становится ближе к прямоугольной.



Разложение через синусы

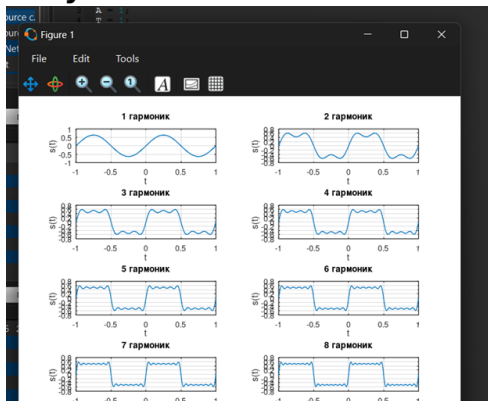
Сценарий был изменён для построения меандра через синусоидальные составляющие.

В разложении использовались только нечётные гармоники с постепенно уменьшающимися амплитудами.

Изменённый сценарий

```
meandr_sin.m
1 N = 8;
2 t = -1:0.01:1;
3 A = 1;
4 T = 1;
5
6 nh = (N+2-1);
7 Am = 2*A*pi ./ nh;
8
9 harmonics = sin(2*pi*nh*t/T);
10 s1 = harmonics .* repmat(Am, 1, length(t));
11 s2 = cumsum(s1);
12
13 figure(3);
14 clf;
15
16 for k = 1:N
17     subplot(4, 2, k);
18     plot(t, s2(k, :));
19     grid on;
20     title(num2str(k), " гармоник");
21     xlabel("t");
22     ylabel("s(t)");
23 end
24
25 drawnow;
26 print("C:/Octave/meandr-sin.png", "-dpng", "-r150");
```

Результат выполнения



Результат разложения

Увеличение количества нечётных гармоник улучшает приближение прямоугольного сигнала.

Около точек разрыва сохраняются колебания, связанные с эффектом Гиббса.

Подготовка каталога spectre1

Для определения спектров был создан каталог spectre1.

В нём был подготовлен сценарий spectre.m для работы с двумя синусоидальными сигналами.

```
octave:6> cd("C:/Octave")
error: invalid character '??' (ASCII 209)

>>> ?? cd("C:/Octave")
      ^

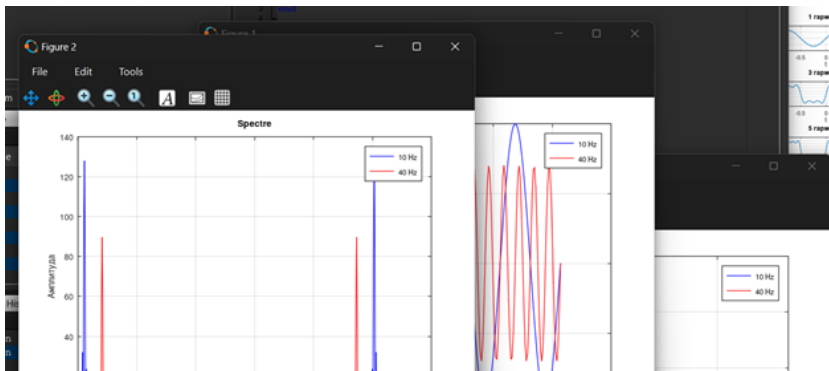
octave:6> cd("C:/Octave")
octave:7> mkdir("spectre1")
octave:8> cd("C:/Octave/spectre1")
octave:9> |
```

Сигналы и их спектры

Были заданы два сигнала:

- ▶ первый сигнал — частота 10 Гц, амплитуда 1;
- ▶ второй сигнал — частота 40 Гц, амплитуда 0,7.

Спектры были вычислены с помощью быстрого преобразования Фурье.



Быстрое преобразование Фурье

Функция `fft` преобразует представление сигнала из временной области в частотную.

На исправленном графике спектра видны пики на частотах 10 Гц и 40 Гц. Эти пики соответствуют частотам исходных сигналов.

Проверка теоремы Котельникова

Частота дискретизации была изменена с 512 Гц на 128 Гц.

Для корректного представления сигнала частотой 40 Гц должно выполняться условие:

$$f_d > 2f_{\max}.$$

В данном случае:

$$128 > 2 \cdot 40.$$

```
4 L
5 if ~exist("spectre", "dir")
6     mkdir("spectre");
7 end
8
9 tmax = 0.5;
10 fd = 128;
```

Press 'Shift+Return' to replace

Что произойдёт при частоте меньше 80 Гц

При частоте дискретизации меньше 80 Гц условие теоремы Котельникова нарушается.

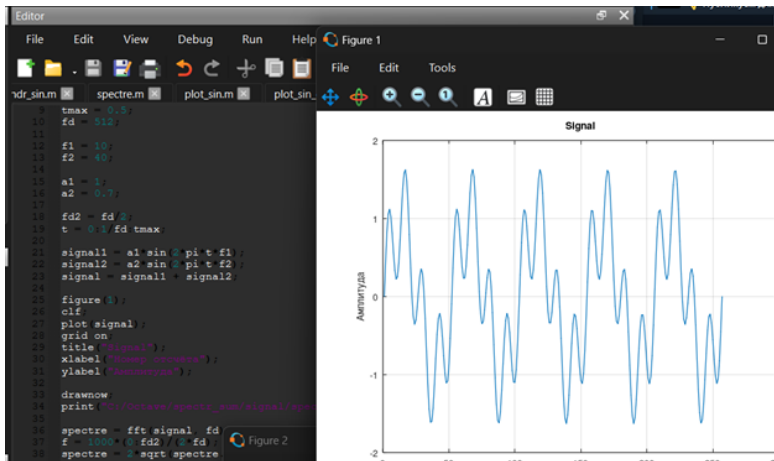
Возникает алиасинг — наложение спектров. Сигнал частотой 40 Гц может отображаться как сигнал с другой, более низкой частотой.

Восстановить исходный сигнал без искажений в этом случае невозможно.

Спектр суммы сигналов

Был создан каталог `spectr_sum` и сценарий `spectre_sum.m`.

В сценарии вычислялась сумма сигналов с частотами 10 Гц и 40 Гц, после чего определялся её спектр.



Результат спектрального анализа

В спектре суммы присутствуют две основные частотные составляющие:

- ▶ 10 Гц;
- ▶ 40 Гц.

Результат соответствует свойству линейности преобразования Фурье: спектр суммы сигналов равен сумме спектров исходных сигналов.

Подготовка сценария модуляции

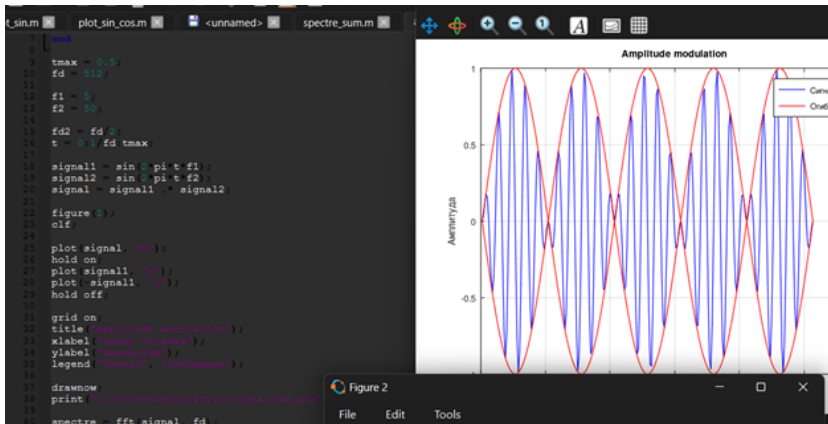
Для моделирования амплитудной модуляции был создан каталог `modulation` и сценарий `am.m`.

```
octave:17> cd("C:/Octave")  
octave:18> mkdir("modification")  
octave:19> mkdir("modulation")  
octave:20> cd("C:/Octave/modulation")  
octave:21>
```

Амплитудно-модулированный сигнал

Информационный сигнал имел частоту 5 Гц, а несущий сигнал — 50 Гц.

Амплитудно-модулированный сигнал был получен перемножением информационного и несущего сигналов.



Спектр модулированного сигнала

В спектре появились две боковые составляющие:

$$f_{\text{нижняя}} = 50 - 5 = 45 \text{ Гц},$$

$$f_{\text{верхняя}} = 50 + 5 = 55 \text{ Гц}.$$

Таким образом, спектр произведения сигналов представляет собой свёртку их спектров.

Установка пакета signal

Для реализации способов кодирования был установлен и подключён пакет `signal`.

Он предоставляет функции, необходимые для дискретизации и обработки сигналов.

```
zeromq | 1.3.7 | ... \mingw64\share\O
5 packages installed
ctave:24> pkg prefix -local
nstallation prefix:          C:\Users\pavli\AppData
s
rchitecture dependent prefix: C:\Users\pavli\AppData
s
ctave:25> pkg install -forge control signal\
kg: downloading latest package database from packag
kg: this may take a moment, please wait...
rror: pkg: could not read URL and no cached package
ternet connection
```

Подготовка каталога coding

Для файлов программы был создан каталог coding.

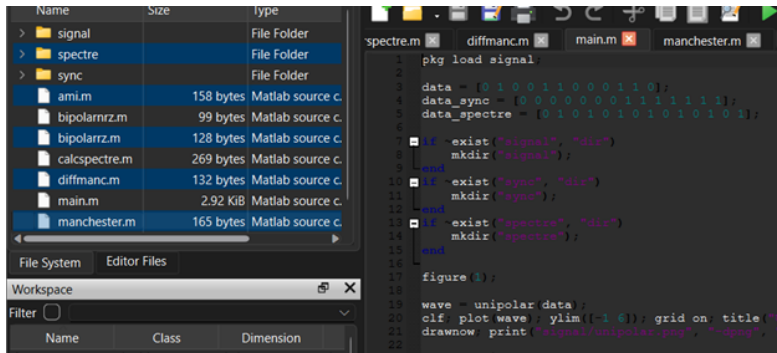
В нём были размещены главный сценарий и отдельные функции кодирования.

```
>>> cd( 'C:/Octave' )  
^  
ctave:27> cd("C:/Octave")  
ctave:28> mkdir("coding")  
ctave:29> cd("C:/Octave/coding")  
ctave:30> |
```

Файлы программы

В каталоге были созданы следующие файлы:

- ▶ main.m и maptowave.m;
- ▶ unipolar.m и ami.m;
- ▶ bipolarnrz.m и bipolarrrz.m;
- ▶ manchester.m и diffmanc.m;
- ▶ calcspectre.m.



Запуск главного сценария

После запуска `main.m` были созданы 18 изображений:

- ▶ шесть графиков кодированных сигналов;
- ▶ шесть графиков для проверки самосинхронизации;
- ▶ шесть графиков спектров.

The screenshot displays the Octave IDE interface. The top-left pane shows a file explorer with a 'signal' folder containing six image files (ami.png, bipolarnr..., bipolarrz..., diffmanc..., manches..., unipolar...) and a 'spectre' folder containing two image files (ami.png, bipolarnr...). The bottom-left pane shows the workspace with a table of variables:

Name	Class	Dimension
A	double	1x1
Am	double	1x8
N	double	1x1
T	double	1x1
a1	double	1x1
a2	double	1x1
ans	double	1x1

The right pane shows the command window with the following output:

```
error: called from
  bipolarnrz at line 2 column 5
octave:33> bipolarrz
error: 'data' undefined near line 2, column 10
error: called from
  bipolarrz at line 2 column 5
octave:34> calcspectre
error: 'wave' undefined near line 5, column 13
error: called from
  calcspectre at line 5 column 5
octave:35> diffmanc
error: 'data' undefined near line 2, column 30
error: called from
  diffmanc at line 2 column 5
octave:36> main
error: 'unipolar' undefined near line 19, column 8
error: called from
  main at line 19 column 1
octave:37> manchester
error: 'data' undefined near line 2, column 10
error: called from
  manchester at line 2 column 5
octave:38> cd("C:/Octave/coding")
error: invalid character '♦♦♦' (ASCII 209)
```

Униполярный код и NRZ

Униполярный код

Единица представляется положительным уровнем, а ноль — нулевым. При длинной последовательности одинаковых битов переходы отсутствуют, поэтому самосинхронизация не обеспечивается.

NRZ

Ноль и единица представлены двумя постоянными уровнями. При длинной последовательности одинаковых битов уровень не изменяется, поэтому синхронизация может быть потеряна.

Код AMI

Нулевой бит представляется нулевым уровнем.

Единицы поочерёдно кодируются положительными и отрицательными импульсами.

Переходы присутствуют при передаче единиц, однако длинная последовательность нулей не содержит переходов. Поэтому AMI не всегда обеспечивает самосинхронизацию.

Коды RZ и Manchester

RZ

Сигнал возвращается к нулевому уровню внутри каждого битового интервала. Благодаря регулярным переходам код обладает самосинхронизацией.

Manchester

В середине каждого битового интервала обязательно происходит изменение уровня. Этот переход позволяет поддерживать синхронизацию передатчика и приёмника.

Дифференциальный Manchester

В дифференциальном манчестерском коде переход уровня обязательно происходит в середине каждого битового интервала.

Значение бита определяется наличием или отсутствием дополнительного перехода в начале интервала.

Код обладает хорошей самосинхронизацией и устойчив к изменению полярности сигнала.

Полученные результаты

- ▶ построены графики тригонометрических функций;
- ▶ выполнено разложение меандра в ряд Фурье;
- ▶ получены спектры отдельных сигналов и их суммы;
- ▶ проверено условие теоремы Котельникова;
- ▶ рассмотрено возникновение алиасинга;
- ▶ смоделирована амплитудная модуляция;
- ▶ реализованы шесть способов линейного кодирования;
- ▶ исследованы свойства самосинхронизации кодов.

Итоги работы

В ходе работы были изучены методы построения, кодирования и модуляции сигналов в GNU Octave.

При увеличении количества нечётных гармоник форма частичного ряда Фурье приближается к меандру. С помощью быстрого преобразования Фурье были определены спектры сигналов с частотами 10 Гц и 40 Гц.

Установлено, что для сигнала частотой 40 Гц частота дискретизации должна быть больше 80 Гц. При нарушении этого условия возникает алиасинг.

В результате исследования линейных кодов установлено, что RZ, манчестерский и дифференциальный манчестерский коды обладают свойством самосинхронизации.

Спасибо за внимание

Спасибо за внимание!

Павличенко Родион Андреевич

НПИбд-02-24